

Báo cáo đồ án cuối kỳ

Khả năng học chính xác thuật toán của mạng nơ-ron

Bài báo gốc:

*Learning to Add, Multiply, and Execute
Algorithmic Instructions Exactly with Neural Networks*

Nhóm 11

23127006 – Trần Nguyễn Khải Luân

23127179 – Nguyễn Bảo Duy

23127333 – Trương Quốc Cường

23127539 – Nguyễn Thanh Tiến

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Giảng viên hướng dẫn: Lê Nhựt Nam

Ngày 7 tháng 1 năm 2026

Nội dung trình bày

- 1 Giới thiệu
- 2 Chứng minh lý thuyết NTK
- 3 Template Matching
- 4 Exact Learnability
- 5 Thực nghiệm & Kết luận
- 6 Tham khảo

Phần 1

Giới thiệu

Câu hỏi nghiên cứu chính

Vấn đề cốt lõi:

- Mạng Neural thường *xấp xỉ* tốt các hàm trơn, nhưng **thất bại** khi tổng quát hóa trên các phép toán rời rạc.
- **Câu hỏi:** Liệu mạng Neural có thể học thực thi các chỉ dẫn thuật toán mã hóa nhị phân **một cách chính xác**?

Phạm vi bài báo:

- Hoán vị nhị phân, Phép cộng, Phép nhân
- Lệnh SBN $\Rightarrow$ **Turing-complete**

Đóng góp chính của bài báo

Hai đối mới quan trọng:

1. **Biểu diễn theo mẫu thuật toán:** Dữ liệu huấn luyện dạng "template số lượng chỉ tăng **logarithmic**.
2. **Tính học được chính xác:** Ensemble mạng 2 lớp ($n_h \rightarrow \infty$) học **chính xác** với xác suất cao.

⇒ **Kết quả NTK-based đầu tiên** cho exact learnability.

Phần 2

Chứng minh lý thuyết NTK

Mục tiêu chung

Mục tiêu lớn của NTK:

- Xét một mạng nơ-ron có chiều rộng ẩn m (số neuron ẩn rất lớn).
- Khi $m \rightarrow \infty$, ta muốn hiểu động lực học của gradient descent (gradient flow)
- Kết quả: trong giới hạn này, việc train mạng nơ-ron

$\Rightarrow$ tương đương với **kernel regression**

với một kernel đặc biệt gọi là **Neural Tangent Kernel**

1. **Output Dynamics:** Gradient flow tuân theo ODE:

$$\dot{\mathbf{f}}(t) = -K(\theta(t))(\mathbf{f}(t) - \mathbf{y})$$

2. **Lazy Training:** Khi $m \rightarrow \infty$, kernel đứng im:

$$K(\theta(t)) \approx \Theta(X, X) \quad (\text{hằng số})$$

3. **Giải ODE tuyến tính:** suy ra Kernel Regression:

$$f_{\infty}(x) = \Theta(x, X)\Theta(X, X)^{-1}\mathbf{y}$$

Thiết lập bài toán

Dữ liệu: $\{(x_i, y_i)\}_{i=1}^n$, với $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$.

Mô hình mạng 2 tầng:

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{r=1}^m a_r \sigma(w_r^\top x)$$

Ký hiệu:

- m : Chiều rộng lớp ẩn.
- $w_r \in \mathbb{R}^d, a_r \in \mathbb{R}$: Trọng số lớp 1 và lớp 2.
- $\theta = (a_r, w_r)_{r=1}^m$: Tập tham số mạng.

Vector Output và Hàm Loss

Vector output và nhãn:

$$\mathbf{f}(\theta) = [f_m(x_1; \theta), \dots, f_m(x_n; \theta)]^\top \in \mathbb{R}^n, \quad \mathbf{y} = [y_1, \dots, y_n]^\top \in \mathbb{R}^n.$$

Hàm mất mát (Mean Squared Error - MSE):

$$L(\theta) = \frac{1}{2} \|\mathbf{f}(\theta) - \mathbf{y}\|_2^2 = \frac{1}{2} \sum_{i=1}^n (f_m(x_i; \theta) - y_i)^2.$$

Mục tiêu 1: Output Dynamics

Gradient flow trên tham số:

$$\frac{d\theta(t)}{dt} = -\nabla_{\theta} L(\theta(t))$$

Tính gradient của Loss $L(\theta) = \frac{1}{2} \|\mathbf{f}(\theta) - \mathbf{y}\|^2$:

$$\nabla_{\theta} L = \nabla_{\theta} \left(\frac{1}{2} (\mathbf{f}(\theta) - \mathbf{y})^{\top} (\mathbf{f}(\theta) - \mathbf{y}) \right) = \nabla_{\theta} \mathbf{f}(\theta)^{\top} (\mathbf{f}(\theta) - \mathbf{y})$$

với $J(\theta) := \nabla_{\theta} \mathbf{f}(\theta) \in \mathbb{R}^{n \times P}$ là ma trận Jacobian.

Kết quả:

$$\frac{d\theta(t)}{dt} = -J(\theta(t))^{\top} (\mathbf{f}(\theta(t)) - \mathbf{y})$$

Mục tiêu 1: Output Dynamics

Chain Rule cho $\mathbf{f}(t) := \mathbf{f}(\theta(t))$:

$$\frac{d\mathbf{f}(t)}{dt} = \underbrace{\nabla_{\theta}\mathbf{f}(\theta(t))}_{J(\theta(t))} \cdot \frac{d\theta(t)}{dt} = J(\theta(t)) \frac{d\theta(t)}{dt}$$

Thay $\frac{d\theta}{dt} = -J^{\top}(\mathbf{f} - \mathbf{y})$ vào biểu thức trên, ta được:

$$\frac{d\mathbf{f}(t)}{dt} = - \underbrace{J(\theta(t))J(\theta(t))^{\top}}_{K(\theta(t))} (\mathbf{f}(t) - \mathbf{y}).$$

Kết luận:

$$\frac{d\mathbf{f}(t)}{dt} = -K(\theta(t))(\mathbf{f}(t) - \mathbf{y})$$

Mục tiêu 2: NTK regime khi $m \rightarrow \infty$

Từ Mục tiêu 1, ta có:

$$\frac{d\mathbf{f}(t)}{dt} = -K_m(\theta_m(t))(\mathbf{f}(t) - \mathbf{y}),$$

trong đó K_m là NTK thực nghiệm của mạng rộng m .

Vấn đề: $K_m(\theta_m(t))$ vẫn phụ thuộc vào t vì tham số $\theta_m(t)$ thay đổi khi train.

Mục tiêu 2: Chứng minh

Chứng minh $K_m(\theta(t)) \rightarrow \Theta(X, X)$ qua 2 bước:

Bước 1: Hội tụ tại khởi tạo

$$K_m(\theta(0)) \xrightarrow[m \rightarrow \infty]{a.s.} \Theta(X, X)$$

Bước 2: Ổn định trong quá trình train

$$\max_{0 \leq t \leq T} \|K_m(\theta(t)) - K_m(\theta(0))\| \xrightarrow{m \rightarrow \infty} 0$$

Kết luận: Kernel đứng im tại giá trị khởi tạo $\Theta(X, X)$.

Phân tích Gradient $g_{ir}(0)$ có bậc $O(1/\sqrt{m})$

Xét neuron r với tham số $\theta_r = (a_r, w_r)$, gradient tại x_i :

$$g_{ir}(0) := \nabla_{\theta_r} f_m(x_i; \theta(0)) = \left(\underbrace{\frac{\sigma(w_r^\top x_i)}{\sqrt{m}}}_{\partial_{a_r}}, \underbrace{\frac{a_r \sigma'(w_r^\top x_i) x_i}{\sqrt{m}}}_{\partial_{w_r}} \right)$$

Kết quả: $\|g_{ir}(0)\| = O(1/\sqrt{m})$.

Đóng góp vào NTK của 1 neuron:

$$\begin{aligned} \langle g_{ir}(0), g_{jr}(0) \rangle &= O(1/m). \\ K_m(\theta(0)) &= \sum_{r=1}^m \langle g_{ir}(0), g_{jr}(0) \rangle = \sum_{r=1}^m O\left(\frac{1}{m}\right) = O(1). \end{aligned}$$

Mục tiêu 2 – Bước 1: luật số lớn cho NTK

Do các neuron r là i.i.d.,

$$K_{m,ij}(0) = \sum_{r=1}^m \langle g_{ir}(0), g_{jr}(0) \rangle \xrightarrow[m \rightarrow \infty]{a.s.} \Theta(x_i, x_j).$$

Kết luận Bước 1:

$$K_m(\theta_m(0)) \xrightarrow[m \rightarrow \infty]{a.s.} \Theta(X, X).$$

Tức là ngay tại khởi tạo, NTK thực nghiệm hội tụ về một kernel không phụ thuộc vào m .

Mục tiêu 2 – Bước 2: tham số đứng im khi train

Ý tưởng của Bước 2: Khi m rất lớn, mỗi tham số chỉ thay đổi rất nhỏ quanh điểm khởi tạo $\theta_m(0)$.

Xét gradient Loss theo tham số neuron r ($\theta_r = (a_r, w_r)$):

$$\nabla_{\theta_r} L = \sum_{i=1}^n (f_i - y_i) \nabla_{\theta_r} f_m(x_i) = \sum_{i=1}^n (f_i - y_i) \underbrace{\frac{1}{\sqrt{m}} \begin{pmatrix} \sigma(w_r^\top x_i) \\ a_r \sigma'(w_r^\top x_i) x_i \end{pmatrix}}_{O(1/\sqrt{m})}$$

$$\Rightarrow \|\nabla_{\theta_r} L\| = O\left(\frac{1}{\sqrt{m}}\right)$$

Mục tiêu 2 – Bước 2: Tham số ổn định

Như đã chứng minh, tốc độ thay đổi gradient là $O(1/\sqrt{m})$.
Tích phân theo thời gian $t \in [0, T]$:

$$\|\theta_r(t) - \theta_r(0)\| \leq \int_0^T \left\| \frac{d\theta_r}{ds} \right\| ds = \int_0^T O\left(\frac{1}{\sqrt{m}}\right) ds = O\left(\frac{T}{\sqrt{m}}\right).$$

Kết luận: Khi $m \rightarrow \infty$, tham số $\theta(t)$ hội tụ về giá trị khởi tạo $\theta(0)$.

Mục tiêu 2 – Bước 2: Gradient ổn định

Bổ đề Lipschitz: Hàm h gọi là Lipschitz (hằng số L) nếu $\|h(x) - h(y)\| \leq L\|x - y\|$. Nếu $\|\nabla h\|$ bị chặn, thì h là Lipschitz.

Áp dụng: Gradient $g_{ir}(\theta)$ có đạo hàm bị chặn $\Rightarrow$ là Lipschitz:

$$\|g_{ir}(t) - g_{ir}(0)\| \leq L\|\theta_r(t) - \theta_r(0)\|$$

Thay kết quả ổn định tham số:

$$\|g_{ir}(t) - g_{ir}(0)\| \leq L \cdot O\left(\frac{T}{\sqrt{m}}\right) = O\left(\frac{T}{\sqrt{m}}\right).$$

Ý nghĩa: Gradient thay đổi rất ít trong quá trình train.

Mục tiêu 2 – Bước 2: NTK ổn định (Kernel đóng băng)

NTK là tổng tích vô hướng: $K_{m,ij}(t) = \sum_{r=1}^m \langle g_{ir}(t), g_{jr}(t) \rangle$.

Xét hiệu số của từng thành phần $\langle g_i(t), g_j(t) \rangle - \langle g_i(0), g_j(0) \rangle$. Áp dụng **Bất đẳng thức tam giác**:

$$\begin{aligned} |\langle g(t), g(t) \rangle - \langle g(0), g(0) \rangle| \\ \leq \|g(t) - g(0)\| \|g(t)\| + \|g(0)\| \|g(t) - g(0)\| \end{aligned}$$

Tổng hợp lại qua m neuron:

$$\max_{t \in [0, T]} \|K_m(t) - K_m(0)\| \leq \sum_{r=1}^m O\left(\frac{1}{m\sqrt{m}}\right) = O\left(\frac{1}{\sqrt{m}}\right) \xrightarrow{m \rightarrow \infty} 0.$$

Bước 2: Tổng hợp tác động của toàn bộ m neuron

Tổng thay đổi của NTK:

$$|K_{m,ij}(t) - K_{m,ij}(0)| = \left| \sum_{r=1}^m (k_{ij}^{(r)}(t) - k_{ij}^{(r)}(0)) \right|.$$

Mỗi hạng tử có bậc $O(1/(m\sqrt{m}))$, nên:

$$|K_{m,ij}(t) - K_{m,ij}(0)| \leq m \cdot O\left(\frac{1}{m\sqrt{m}}\right) = O\left(\frac{1}{\sqrt{m}}\right) \xrightarrow{m \rightarrow \infty} 0.$$

Kết luận Bước 2:

$$\|K_m(\theta_m(t)) - K_m(\theta_m(0))\| = O\left(\frac{1}{\sqrt{m}}\right) \rightarrow 0.$$

Mục tiêu 2 – Kết luận NTK regime

Gộp Bước 1 và Bước 2:

- (Bước 1) Tại khởi tạo,

$$K_m(\theta_m(0)) \xrightarrow[m \rightarrow \infty]{a.s.} \Theta(X, X).$$

- (Bước 2) Trong quá trình train với thời gian hữu hạn,

$$\sup_{t \in [0, T]} \|K_m(\theta_m(t)) - K_m(\theta_m(0))\| \xrightarrow{m \rightarrow \infty} 0.$$

Kết luận: Trong NTK regime, ta có xấp xỉ:

$$\frac{d\mathbf{f}(t)}{dt} = -K_m(\theta_m(t))(\mathbf{f}(t) - \mathbf{y}) \approx -\Theta(X, X)(\mathbf{f}(t) - \mathbf{y}).$$

Mục tiêu 3: Nghiệm ODE và Hội tụ

Thay kernel đúng im $K(t) \approx \Theta(X, X)$ vào ODE mục tiêu 1:

$$\frac{d}{dt}(\mathbf{f}(t) - \mathbf{y}) = -\Theta(X, X)(\mathbf{f}(t) - \mathbf{y})$$

Đây là ODE tuyến tính với nghiệm giải tích:

$$\mathbf{f}(t) = \mathbf{y} + e^{-\Theta(X, X)t}(\mathbf{f}(0) - \mathbf{y})$$

Kết quả hội tụ: Khi $t \rightarrow \infty$ (với $\lambda_{\min}(\Theta) > 0$):

$$\mathbf{f}(t) \rightarrow \mathbf{y} \implies \text{Loss} \rightarrow 0$$

Dự đoán tại điểm test (NTK Predictor)

Với điểm test x , dynamics tuân theo: $\frac{df_t(x)}{dt} = -\Theta(x, X)(\mathbf{f}(t) - \mathbf{y})$.
Giải phương trình và giả sử khởi tạo nhỏ ($f_0 \approx 0$), ta có kết quả cuối cùng:

$$f_{\infty}(x) = \Theta(x, X)\Theta(X, X)^{-1}\mathbf{y}$$

Ý nghĩa quan trọng:

- Việc train mạng nơ-ron vô hạn (GD) tương đương với giải bài toán Kernel Regression với kernel Θ .
- Ta có thể tính toán kết quả mà không cần train mạng!

Tóm tắt toàn bộ chứng minh NTK

Mục tiêu 1: Từ gradient flow và Jacobian:

$$\dot{\mathbf{f}}(t) = -K(\theta(t))(\mathbf{f}(t) - \mathbf{y}), \quad K = JJ^\top.$$

Mục tiêu 2: (NTK regime) Khi width $m \rightarrow \infty$:

$$K_m(\theta(t)) \approx K_m(\theta(0)) \rightarrow \Theta(X, X),$$

Mục tiêu 3: Thay K bằng $\Theta(X, X)$, giải ODE:

$$f_\infty(x) = \Theta(x, X)\Theta(X, X)^{-1}\mathbf{y},$$

$\Rightarrow$ gradient descent trên mạng rộng = **kernel regression** với NTK.

Kết quả kinh điển: NNGP Kernel của ReLU

Kết quả cổ điển từ Cho & Saul (2009), Neal (1996), dùng lại trong Jacot et al. (2018) và Lee et al. (2019):

Khi $w \sim \mathcal{N}(0, \frac{1}{k'}I)$, ta có kernel NNGP:

$$K(x, x') = \frac{\|x\| \|x'\|}{2\pi k'} \left((\pi - \theta) \cos \theta + \sin \theta \right) I_k$$

trong đó

$$\cos \theta = \frac{x^\top x'}{\|x\| \|x'\|}.$$

Đây là **arc-cosine kernel cấp 1 (ReLU NNGP kernel)**.

Kết quả kinh điển: NTK của mạng hai tầng ReLU

Theo Jacot et al. (2018), Lee et al. (2019): khi chiều rộng $n_h \rightarrow \infty$, NTK thực nghiệm hội tụ về kernel xác định:

$$\Theta(x, x') = \frac{x^\top x'}{2\pi k'}(\pi - \theta) + \frac{\|x\| \|x'\|}{2\pi k'} \left((\pi - \theta) \cos \theta + \sin \theta \right) I_k$$

với cùng định nghĩa

$$\cos \theta = \frac{x^\top x'}{\|x\| \|x'\|}.$$

NTK này gồm:

- một phần tử đạo hàm theo W_2 (ReLU kernel),
- một phần tử đạo hàm theo W_1 (term $x^\top x'$).

Phần 3

Template Matching

Nguyên lý Template Matching

Ý tưởng:

- Thuật toán được phân rã thành các **template** thao tác trên từng block bit.
- Mỗi template: (x, y) input block $\rightarrow$ output vector.
- Thực thi = **matching** input với template đã học.

Cấu trúc Block:

- Input $\hat{x} \in \{0, 1\}^k$ chia thành b blocks.
- Tổng số template: $O(b) \ll O(2^b)$.

Hàm Template Matching

Với mỗi block B_i :

$$f_i(x') = \begin{cases} y & \text{nếu } (x', y) \in T_i \\ \mathbf{0} & \text{ngược lại} \end{cases}$$

Hàm tổng hợp: $f(\hat{x}) = \bigvee_{i=1}^b f_i(\hat{x}[B_i])$

Thực thi lặp: $\hat{x}_{t+1} = f(\hat{x}_t)$ cho đến trạng thái ổn định.

Ví dụ: Binary Addition

Thiết lập cho 2 số ℓ -bit:

- 2ℓ blocks: (p_i, q_i) và carry c_i
- Templates: bitwise sum + carry propagation

Thông kê:

- Số templates: 4ℓ (tuyến tính)
- Số bước lặp: 2ℓ

Ví dụ: Binary Multiplication

Mô phỏng Shift-and-Add:

1. Kiểm tra LSB multiplier
2. Cộng multiplicand vào tổng
3. Shift multiplicand trái, multiplier phải

Thống kê:

- Số templates: 21ℓ
- Worst-case: $4\ell^2 + 3\ell$ iterations

Phần 4

Exact Learnability

Phần I: Huấn luyện với dữ liệu trực chuẩn hoá

Mã hóa Template thành Training Data:

- Template $(x^{(i,j)}, y^{(i,j)}) \in T_i \rightarrow$ vector $q_{ij} \in \mathbb{R}^{k'}$
- q_{ij} có cấu trúc block-partitioned: chỉ block i khác 0
- Các q_{ij} được **trực giao hóa** trong mỗi block

$\Rightarrow$ NTK matrix $\Theta(X, X) = \text{scaled identity} + \text{rank-1 noise}$

Ví dụ minh họa: Phân rã thành các Luật Cục bộ

Thay vì học bảng cộng cho toàn bộ chuỗi bit (ví dụ $011 + 101$), thuật toán được phân rã thành các **thao tác cục bộ** tại mỗi vị trí i :

- **Nhóm 1: Bitwise Addition (Tại bit i)**

- Quy tắc A_i : $(p_i = 1, q_i = 0) \rightarrow p_i = 1$
- Quy tắc B_i : $(p_i = 1, q_i = 1) \rightarrow p_i = 0$, sinh bit nhớ $c_i = 1$

- **Nhóm 2: Carry Propagation (Tại bit i)**

- Quy tắc C_i : Nếu có bit nhớ $c_i = 1 \rightarrow$ cộng dồn vào q_{i+1} (hoặc lan truyền tiếp).

Chiến lược Encode: "Bag of Rules"

Tổng hợp Templates: Với chuỗi L bit, ta có tập hợp tất cả các quy tắc cục bộ cho mọi i :

$$T_{total} = \bigcup_{i=0}^{L-1} \{\text{Rules tại } i\}$$

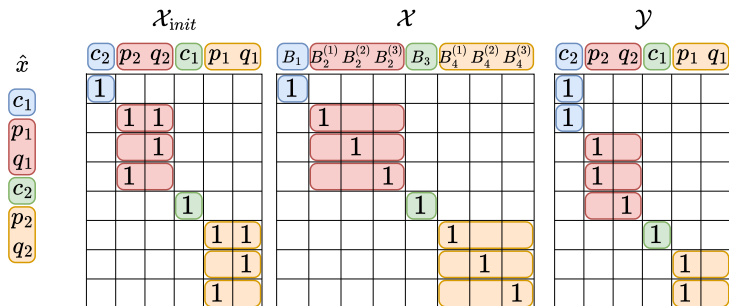
Mã hóa (Encoding): Ta gộp tất cả N rules cục bộ vào một không gian vector chung $\mathbb{R}^N$. Mỗi rule $t_k \in T_{total}$ được one-hot encoding.

Training Data (Identity Matrix):

$$X_{train} = I_N = \begin{bmatrix} e_1 & e_2 & \dots & e_N \end{bmatrix}$$

$\Rightarrow$ Mạng học song song tất cả các quy tắc cục bộ.

Minh họa cụ thể cho Binary Addition



Cấu trúc dữ liệu đầu vào thử nghiệm

Cấu trúc: $\hat{x} = \frac{1}{\sqrt{n_{\hat{x}}}}(\hat{x}_1, \dots, \hat{x}_b)^\top$

- Mỗi block $\hat{x}_i = 0$ hoặc match một sample trong X
- $n_{\hat{x}}$ = số blocks active

Quan trọng:

- Training: $O(b)$ samples
- Test: $O(2^b)$ possible inputs

Theorem 5.1: NTK Predictor Behavior

Assumption 5.1: Số tương quan không mong muốn $<$ tỷ lệ $-w_1/w_0$

Theorem: Dưới Assumption 5.1, NTK predictor bảo toàn dấu:

$$\begin{cases} \mu(\hat{x})_i \leq 0 & \text{nếu } f(\hat{x})_i = 0 \\ \mu(\hat{x})_i > 0 & \text{nếu } f(\hat{x})_i = 1 \end{cases}$$

$\Rightarrow$ **Rounding** $\mu(\hat{x})_i$ cho kết quả chính xác!

Proof Sketch của Theorem 5.1

NTK predictor có dạng:

$$\mu(\hat{x})_i = \sum_{(j,l) \in I^+} f(q_{jl})_i \cdot w_1 + \sum_{(j,l) \in I^-} f(q_{jl})_i \cdot w_0$$

Trong đó:

- I^+ : training samples match $\hat{x}$ ($w_1 > 0$)
- I^- : training samples không match ($w_0 \leq 0$)

Assumption 5.1: $|I_1^-| < -w_1/w_0 \Rightarrow$ dấu được bảo toàn.

Phần 2: Ensemble Complexity

Vấn đề: Theorem 5.1 đúng cho kỳ vọng $\mu(\hat{x})$.
Thực tế: mỗi model có variance $\sigma^2(\hat{x})$.

Giải pháp: Ensemble N models:

$$G(\hat{x}) = \frac{1}{N} \sum_{j=1}^N F^j(\hat{x})$$

Rounding $G(\hat{x})$ cho xác suất cao nếu N đủ lớn.

Công thức Ensemble Complexity (Equation 8)

Bound:

$$N \geq 8 \max_{\hat{x}, i \in [k]} \left\{ \frac{\sigma^2(\hat{x})}{\mu^2(\hat{x})_i} \right\} \ln \left(\frac{2k'}{\delta} \right)$$

Với xác suất $1 - \delta$, rounding ensemble mean cho kết quả **chính xác**.

Lemma 6.1: Asymptotic Orders & Union Bound

- **Phân tích tiệm cận (Lemma 6.1):**

- Variance: $\sigma^2(\hat{x}) \in O(1/k')$ [cite: 489]
- Mean: $|\mu(\hat{x})_i|$ đạt tối thiểu $\Theta(1/\sqrt{k'})$ khi $n_{\hat{x}} = ck'$
- Tỷ lệ: $\frac{\sigma^2}{\mu^2} \in O(k')$

- **Từ 1 input đến toàn bộ không gian (2^b inputs):**

- Áp dụng **Union Bound** cho tất cả các trường hợp và m bước lặp:
 - $N \geq 8 \max \frac{\sigma^2}{\mu^2} \ln\left(\frac{2k' \cdot 2^b \cdot m}{\delta}\right) \approx O(k'b + k' \log m)$

- **Ý chính:** Khi xét toàn bộ thuật toán, thành phần $k'b$ trở thành hạng tử chủ đạo.

Remark 6.1: Kết quả Ensemble Complexity (N)

Độ phức tạp cho các tác vụ với số bit ℓ :

Tác vụ	b	k'	m (lter)	Complexity N
Permutation	ℓ	ℓ	1	$O(\ell^2)$
Addition	2ℓ	4ℓ	2ℓ	$O(\ell^2)$
Multiplication	11ℓ	21ℓ	$4\ell^2 + 3\ell$	$O(\ell^2)$

- Giải thích:**

- Trong mọi tác vụ, k' và b đều tỷ lệ tuyến tính với ℓ .
- $N \in O(\ell \cdot \ell + \ell \log \ell) \Rightarrow$ **Bậc cao nhất là $O(\ell^2)$.**

Phần 5

Thực nghiệm & Kết luận

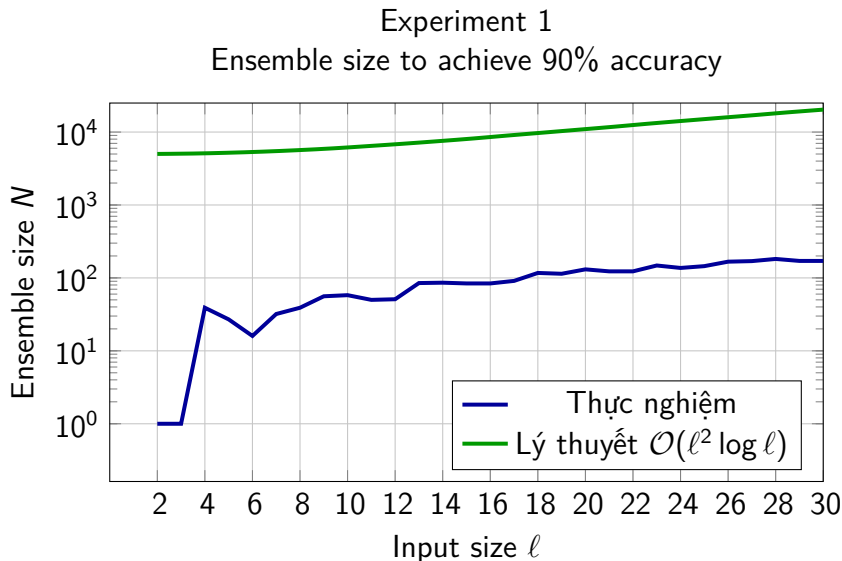
Validation thực nghiệm (Paper Figure 5)

Thiết lập:

- Mạng 2 lớp, 50,000 nút ẩn (hidden units)
- Full-batch gradient descent
- Bit length ℓ từ 2 đến 30

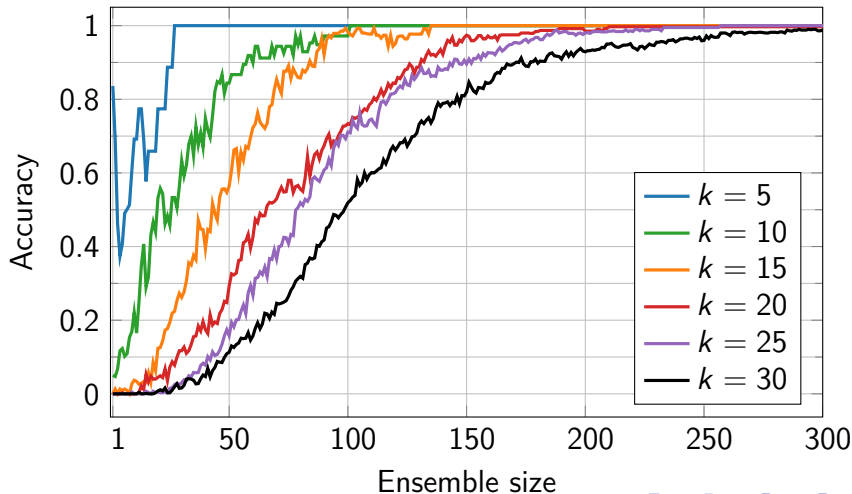
Kết quả: Ensemble complexity tăng theo $O(\ell^2 \log \ell)$ như lý thuyết dự đoán.

Paper Figure 5: Experiment 1

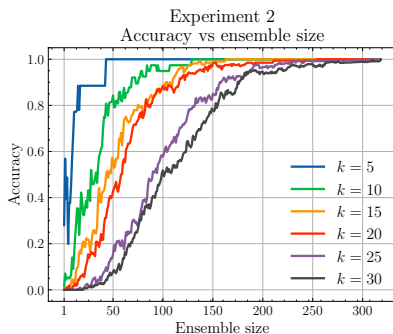
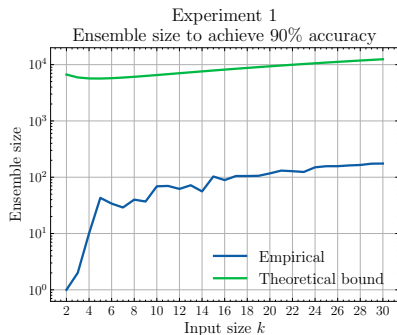


Thí nghiệm 2: Accuracy vs ensemble size

Experiment 2
Accuracy vs ensemble size



Kết quả thí nghiệm (từ bài báo)



Điều bài báo đã chứng minh:

- Mạng 2 lớp ($n_h \rightarrow \infty$) học **chính xác** lệnh thuật toán nhị phân.
- Chỉ cần $O(b)$ training samples (logarithmic).
- Ensemble: $N \in O(\ell^2 \log \ell)$.

Ý nghĩa:

- Kết quả NTK-based đầu tiên cho exact learnability.
- SBN Turing-complete $\Rightarrow$ mở rộng cho mọi hàm tính được.

Hạn chế và hướng phát triển

Hạn chế:

- Giả định dữ liệu trực giao (tính orthogonality giữa các mẫu huấn luyện).
- Bộ nhớ bị giới hạn (cần huấn luyện lại khi kích thước đầu vào thay đổi).

Hướng phát triển:

- Mở rộng sang trường hợp các mẫu huấn luyện có tương quan.
- Nghiên cứu áp dụng GNN trên đồ thị có bậc bị chặn (bounded degree).
- Mở rộng cho Transformer.

Cảm ơn vì đã lắng nghe!

Q&A

Tài liệu tham khảo I



Luca, Artur Back de, George Giapitzakis **and** Kimon Fountoulakis (2025). Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks. **in**. arXiv: 2502.16763 [cs.LG]. URL: <https://arxiv.org/abs/2502.16763>.



Jacot, Arthur, Franck Gabriel **and** Clément Hongler (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. **in** *Advances in Neural Information Processing Systems (NeurIPS)*. URL: <https://arxiv.org/abs/1806.07572>.



Lee, Jaehoon **and others** (2018). Deep Neural Networks as Gaussian Processes. **in** *International Conference on Learning Representations (ICLR)*.

Tài liệu tham khảo II



Yang, Greg (2019). Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes. **in** arXiv: 1910.12478. URL: <https://arxiv.org/abs/1910.12478>.



Chizat, Lénaïc **and** Francis Bach (2018). On the Global Convergence of Gradient Descent for Over-Parameterized Models using Optimal Transport. **in** *Advances in Neural Information Processing Systems (NeurIPS)*. Extended version: arXiv:1805.09545 (2019).



Rahimi, Ali **and** Benjamin Recht (2008). Random Features for Large-Scale Kernel Machines. **in** *Advances in Neural Information Processing Systems (NeurIPS)*.

Tài liệu tham khảo III

-  Williams, Christopher K. I. (1997). Computing with Infinite Networks. *in Advances in Neural Information Processing Systems (NeurIPS)*.
-  Cho, Youngmin **and** Lawrence K. Saul (2009). Kernel Methods for Deep Learning. *in Advances in Neural Information Processing Systems (NeurIPS)*.
-  Schölkopf, Bernhard **and** Alexander J. Smola (2002). Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond. MIT Press.
-  Arora, Sanjeev **and others** (2019). On Exact Computation with an Infinitely Wide Neural Net. *in Advances in Neural Information Processing Systems (NeurIPS)*.

Tài liệu tham khảo IV

-  Du, Simon S. **and others** (2019). Gradient Descent Finds Global Minima of Deep Neural Networks. *in International Conference on Machine Learning (ICML)*.
-  Mei, Song, Andrea Montanari **and** Phan-Minh Nguyen (2018). A Mean Field View of the Landscape of Two-Layer Neural Networks. *in Proceedings of the National Academy of Sciences (PNAS)* 115.33, E7665–E7671.
-  Vaswani, Ashish **and others** (2017). Attention Is All You Need. *in Advances in Neural Information Processing Systems (NeurIPS)*.
-  Zaremba, Wojciech **and** Ilya Sutskever (2014). Learning to Execute. *in*. arXiv: 1410.4615. URL: <https://arxiv.org/abs/1410.4615>.

Tài liệu tham khảo V

-  Graves, Alex, Greg Wayne **and** Ivo Danihelka (2014). Neural Turing Machines. **in**. arXiv: 1410.5401. URL: <https://arxiv.org/abs/1410.5401>.
-  Graves, Alex, Greg Wayne, Malcolm Reynolds **and others** (2016). Hybrid Computing Using a Neural Network with Dynamic External Memory. **in** *Nature* 538, **pages** 471–476.
-  Kaiser, Łukasz **and** Ilya Sutskever (2015). Neural GPUs Learn Algorithms. **in**. Presented at ICLR 2016. arXiv: 1511.08228. URL: <https://arxiv.org/abs/1511.08228>.
-  Reed, Scott **and** Nando de Freitas (2016). Neural Programmer-Interpreters. **in** *International Conference on Learning Representations (ICLR)*.

Tài liệu tham khảo VI

-  Devlin, Jacob **and others** (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. **in** *NAACL-HLT*. arXiv:1810.04805 (2018). URL: <https://arxiv.org/abs/1810.04805>.
-  Brown, Tom B. **and others** (2020). Language Models are Few-Shot Learners. **in** *Advances in Neural Information Processing Systems (NeurIPS)*.
-  Cobbe, Karl **and others** (2021). Training Verifiers to Solve Math Word Problems. **in**. Dataset: GSM8K. arXiv: 2110.14168. URL: <https://arxiv.org/abs/2110.14168>.
-  Yun, Chulhee **and others** (2020). Are Transformers Universal Approximators of Sequence-to-Sequence Functions? **in** *International Conference on Learning Representations (ICLR)*.

Tài liệu tham khảo VII

-  Pérez, Jorge, Pablo Barceló **and** Javier María Tur (2021). The Computational Power of Transformers. **in** *International Conference on Learning Representations (ICLR)*.
-  Hopcroft, John E. **and** Jeffrey D. Ullman (1979). Introduction to Automata Theory, Languages, and Computation. *Addison-Wesley*.
-  Minsky, Marvin (1967). Computation: Finite and Infinite Machines. *Prentice-Hall*.
-  Siegelmann, Hava T. **and** Eduardo D. Sontag (1995). On the Computational Power of Neural Nets. **in** *Journal of Computer and System Sciences* 50.1, **pages** 132–150.
-  Turing, Alan M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem. **in** *Proceedings of the London Mathematical Society* 42.2, **pages** 230–265.

Tài liệu tham khảo VIII



Cook, Stephen A. (1971). The Complexity of Theorem-Proving Procedures. *in Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*.